STEP-BY-STEP VISUAL GUIDE

The Transformer: Encoder and Decoder, Flow by Flow

How a sentence travels through the network, drawn as diagrams and followed with real numbers from a real translation model.

Worked example

A visitor in Mysuru asks, in English:

"I am going to Mysuru tomorrow."

The Transformer answers in Kannada: ನಾಳೆ ನಾನು ಮೈಸೂರಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ.

Inside: the big architecture diagram, a 13-step flow, one encoder layer and one decoder layer opened up, the attention matrices of the real model, the token-by-token decoding loop, and how training differs from use.

Numbers are from NLLB-200 (distilled 600M, Meta AI): 12 encoder + 12 decoder layers, width 1024, 16 attention heads, 615 million parameters. Diagrams are original.

1. The job and the two halves

REAL-WORLD PICTURE

An interpreter team at the Mysuru bus stand. The **reader** (encoder) listens to the whole English sentence once and writes detailed notes on every word, including how the words relate. The **writer** (decoder) then produces the Kannada *one word at a time*; before each word she re-reads the reader's notes and her own sentence so far.

Encoder: understands

Input: all source words together. Output: one vector (1024 numbers) per word, each already informed by every other word. Runs **once**.

Decoder: writes

Input: the words written so far + the encoder's notes. Output: probabilities for the next word. Runs **once per output word**, in a loop.

Our example in tokens

Models do not read words; they read **tokens** (word pieces) and each token has an ID. The real NLLB tokenizer cuts the sentence into 10 pieces:

`eng_Latn` `_I` `_am` `_going` `_to` `_Mys` `uru` `_tomorrow` `.` `</s>`

"_" marks the start of a word. "Mys" + "uru" shows a rare name split into pieces; `eng_Latn` tells the model the source language and `</s>` marks the end. IDs: 256047, 117, 259, 17221, 202, 23059, 1027, 103005, 248075, 2.

The Kannada answer is also written in pieces, one per loop turn (11 tokens including the end marker):

`_ನಾ` `ಳೆ` `_ನಾನು` `_ಮ` `ೈಸ` `ೂರ` `ಿಗೆ` `_ಹೋಗ` `ುತ್ತಿದ್ದೇನೆ` `.` `</s>`

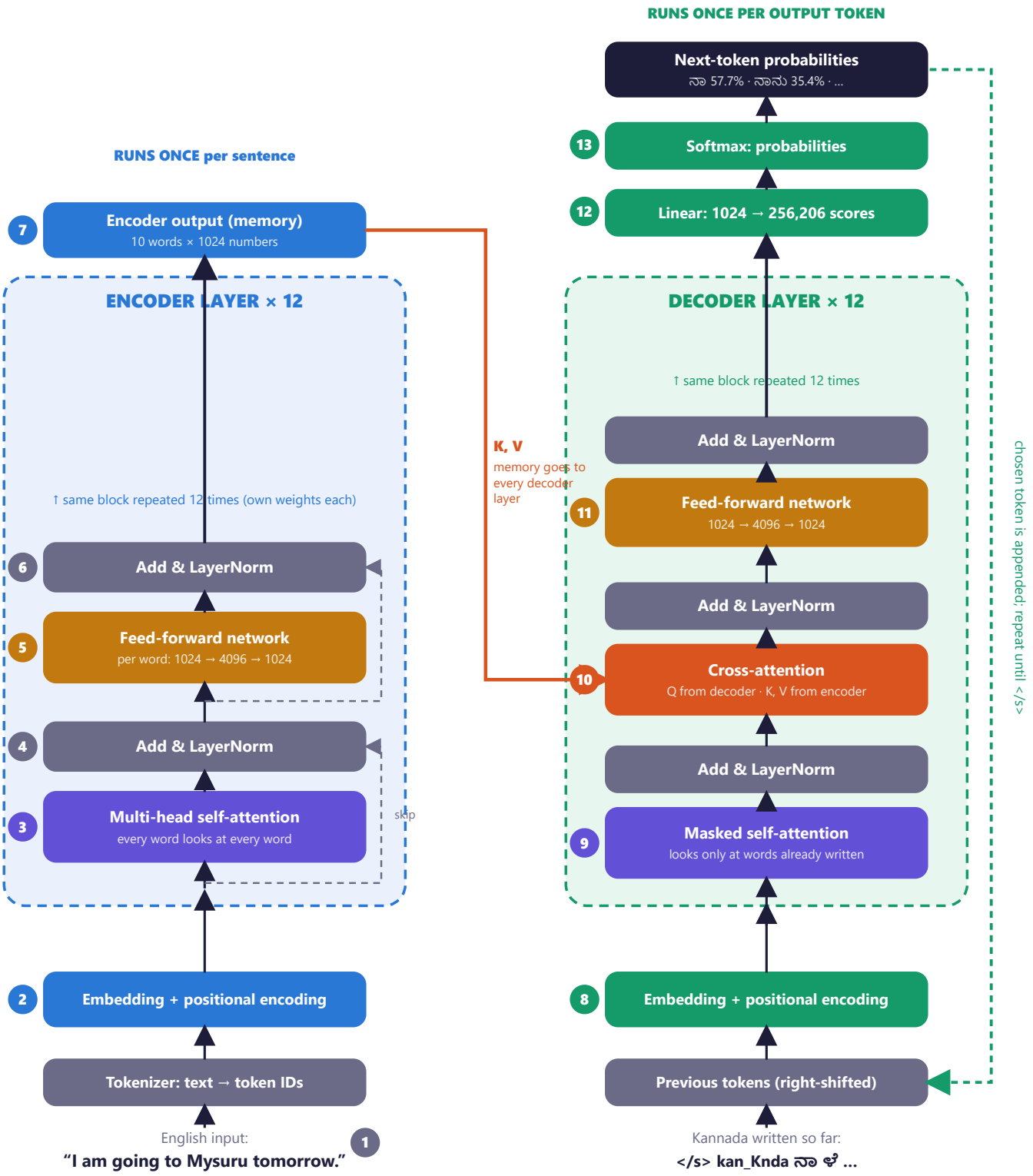


The same two halves, used alone or together: this guide follows the full encoder + decoder (translation).

READING THE DIAGRAMS

Blue = encoder, **green** = decoder, **purple** = self-attention, **orange** = cross-attention (decoder looks at the encoder), **amber** = feed-forward, grey = add & normalise and plumbing. Numbered circles match the 13 steps on the next pages.

2. The complete architecture



Encoder (left) runs once; its output is given to every decoder layer (orange arrow). The decoder (right) runs in a loop, feeding each chosen token back in. “Add & LayerNorm” is drawn as in the 2017 paper; NLLB applies the LayerNorm before each sub-layer instead (see the layer diagrams).

3. The whole flow in 13 steps



IN PLAIN WORDS

Phase A turns the English sentence into "understood notes" (steps 1 to 7). Phase B uses those notes to write Kannada token by token (steps 8 to 13). Steps 8 to 13 repeat until the model chooses the end token. The encoder is never run again.

#	What happens	Shape in the real model
1	Text is cut into tokens and each gets an ID	10 integers
2	IDs become vectors (embedding) and a position signal is added	10 × 1024
3	Self-attention: each word gathers information from all words	10 × 1024 (16 heads of 64)
4	Add the skip connection, normalise	unchanged
5	Feed-forward network, word by word	1024 → 4096 → 1024
6	Add skip, normalise; steps 3 to 6 form one layer, repeated 12 times	10 × 1024
7	Encoder output = memory handed to the decoder	10 × 1024
8	Kannada tokens so far: embed + position	t × 1024
9	Masked self-attention over the Kannada written so far	t × 1024
10	Cross-attention: look at the encoder memory	t × 1024
11	Feed-forward (each decoder layer has steps 9 to 11, repeated 12 times)	t × 1024
12	Linear layer + softmax over the whole vocabulary (last position only is needed)	256,206 probabilities
13	Choose a token (greedy: the most probable), append, loop	t grows by 1

4. Steps 1 and 2: tokens become vectors, plus position

IN PLAIN WORDS

The network only handles numbers. Each token ID selects a row of 1024 learned numbers (its **embedding**), a point in a space where words with related meaning sit near each other. But the network looks at all words at the same time, so it would not know word order; therefore a **position signal** is added so that "Mysuru" as word 6 differs from "Mysuru" as word 2.

THE MATHS

For token ID x_i at position i, the input to layer 1 is

$$X_i = \sqrt{d} \cdot E[x_i] + PE(i), \quad d = 1024, \sqrt{d} = 32$$

$$PE(pos, 2k) = \sin(\, pos \,/\, 10000^{2k/d}) \quad PE(pos, 2k{+}1) = \cos(\, pos \,/\, 10000^{2k/d})$$

Each dimension pair is a sine/cosine wave of a different wavelength, so every position gets a unique pattern and nearby positions get similar patterns. (NLLB stores all the sines in the first half of the 1024 numbers and the cosines in the second half; same idea.)

Real numbers (first 4 of 1024 dimensions)

#	token	ID	embedding (scaled), first 4	after adding position, first 4
1	eng_Latn	256047	-5.00, -1.27, -0.94, -0.83	-4.09, -0.35, +0.00, +0.12
2	I	117	-0.69, -0.38, +0.10, +0.01	-0.55, -0.18, +0.35, +0.30
3	am	259	+1.39, -0.03, -0.02, +0.17	+0.63, -0.73, -0.67, -0.43
4	going	17221	-2.19, +0.32, +0.17, -0.62	-3.15, -0.66, -0.82, -1.62
5	to	202	-4.54, -0.23, +0.06, +0.13	-4.82, -0.61, -0.42, -0.44
6	Mys	23059	-2.90, +1.73, +0.04, -0.42	-2.25, +2.28, +0.49, -0.07
7	uru	1027	-3.80, +0.39, -0.26, -0.73	-2.81, +1.39, +0.73, +0.23
8	tomorrow	103005	-2.95, +0.12, +0.68, -2.93	-2.54, +0.67, +1.36, -2.15
9	.	248075	-2.49, -0.28, +0.01, -0.28	-3.03, -0.66, -0.21, -0.33
10	</s>	2	-0.78, -0.91, -0.94, -0.96	-1.78, -1.89, -1.87, -1.80

Compare the last two columns: the position signal shifts each vector. Both columns come from the real model's input layer (positions here use the model's own sinusoidal table).

REAL-WORLD PICTURE

Each word is written on a card with 1024 attributes; then a numbered sticker (its position) is stuck on the card so the card keeps its place in the line even after the cards are shuffled into one pile.

5. Steps 3 to 7: inside an encoder layer

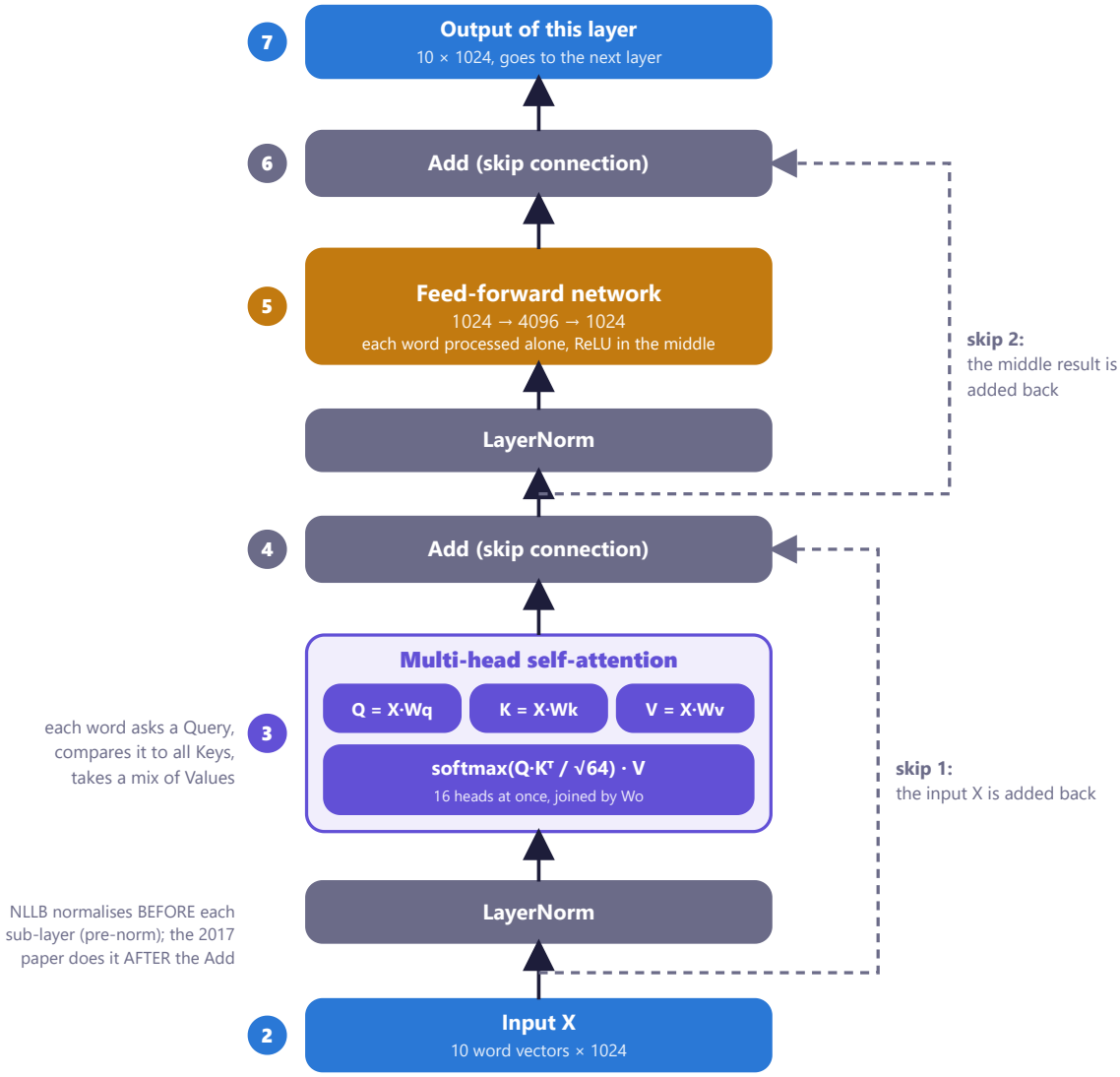


Diagram of one encoder layer. Skip connections (grey) add each sub-layer's input back to its output.

STEP 3: SELF-ATTENTION, IN PLAIN WORDS

Every word asks a question (**Q**uery), every word offers a label (**K**ey) and content (**V**alue). A word compares its query with all keys; the better the match, the more of that word's value it takes. After this, "Mysuru" carries information about "going" and "tomorrow".

THE MATHS

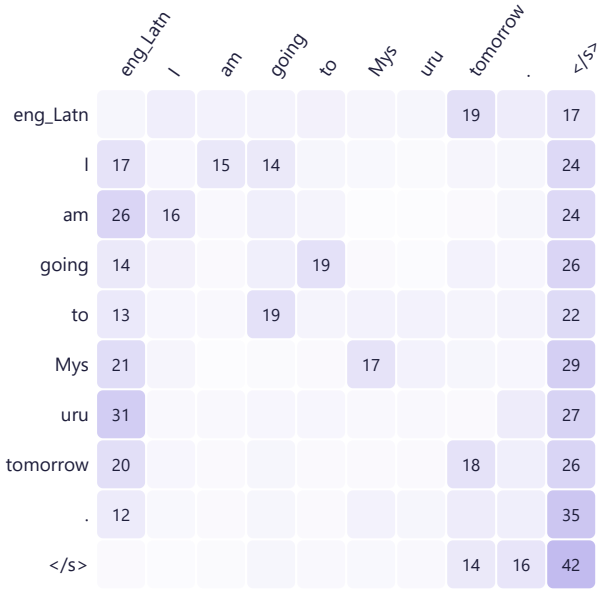
With X the 10 × 1024 input, learned matrices W_Q, W_K, W_V (1024 × 1024):

$$Q = X W_Q \quad K = X W_K \quad V = X W_V \quad \text{Attention} = \text{softmax}(Q K^T / \sqrt{d_k}) \cdot V$$

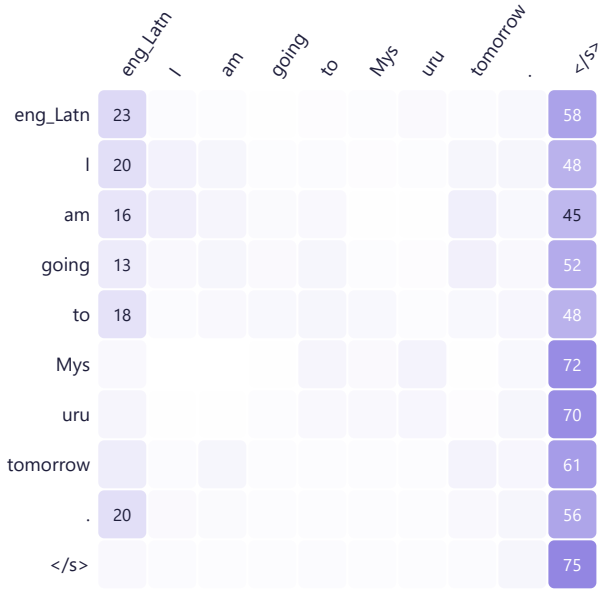
The split into 16 heads gives each head d_k = 64; each head learns its own kind of relationship; the 16 results are joined and mixed by one more matrix. **Step 4 (skip):** $Y = X$ + Attention(LayerNorm(X)). **Step 5:** FFN(u) = W_2 ReLU($W_1 u$ + b_1) + b_2 with 1024 → 4096 → 1024, applied to each word separately. **Step 6 (skip):** $Z = Y$ + FFN(LayerNorm(Y)). The 2017 paper normalises after the Add instead (post-norm); NLLB, like most recent models, normalises before each sub-layer (pre-norm). Both use the same ingredients.

6. The real encoder attention for our sentence

These are the actual attention weights (average over the 16 heads, in percent) of the real model on “I am going to Mysuru tomorrow.” Row = the word that is looking; column = the word it looks at; each row adds up to 100.



Encoder layer 1 (first layer)



Encoder layer 12 (last layer)

Where each word looks most (excluding itself and the markers)

word	layer 1 looks at	layer 12 looks at
I	am (15%)	. (6%)
am	I (16%)	tomorrow (10%)
going	to (19%)	tomorrow (9%)
to	going (19%)	. (5%)
Mys	uru (8%)	uru (7%)
uru	. (11%)	. (6%)
tomorrow	. (10%)	am (5%)
.	tomorrow (12%)	tomorrow (4%)

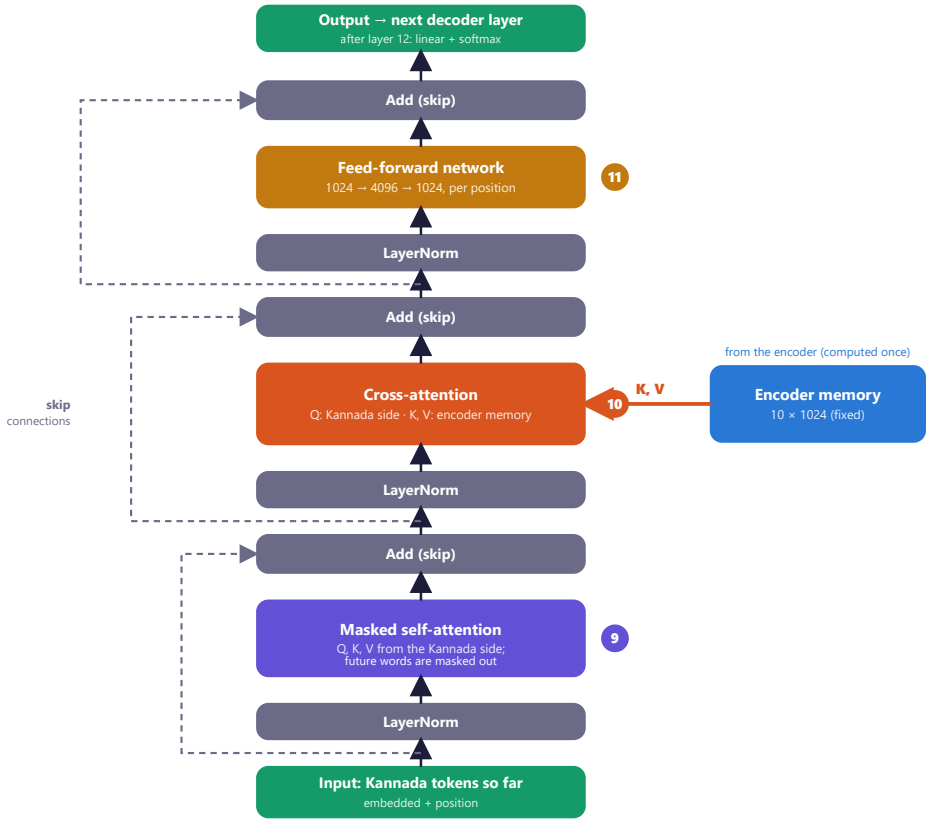
HOW TO READ THIS HONESTLY

This is an average over 16 heads, so it blurs different specialisations. Early layers lean on neighbouring words and the end marker; later layers mix more widely. Attention maps show where information *can* flow; they are not a full explanation of the model’s reasoning. A large share often sits on the end marker `</s>` and the language tag, which act as “rest” positions.

STEP 7: THE ENCODER OUTPUT (THE MEMORY)

Think of twelve rounds of a meeting: in each round every word listens to all the others (attention), then thinks on its own (feed-forward). After round 12 the 10 vectors (10 × 1024) are the **memory**: computed once, kept, and read by the decoder in every layer at every loop turn.

7. Steps 8 to 11: inside a decoder layer



One decoder layer: masked self-attention, cross-attention to the encoder memory, feed-forward.

STEP 9: MASKED SELF-ATTENTION

The decoder is writing left to right, so when it works on word t it may only look at words 1…t. Looking at later words would be cheating during training. A **mask** blocks them: their scores are set to $-\infty$ before the softmax so their weight is 0.

$$\text{softmax}(\, Q\,K^{T} / \sqrt{d_k} + M)\; V, \quad M_{ij} = 0 \text{ if } j \le i, \text{ else } -\infty$$

who may look at whom (rows look at columns)



× = blocked (score set to $-\infty$ before softmax, so weight = 0)

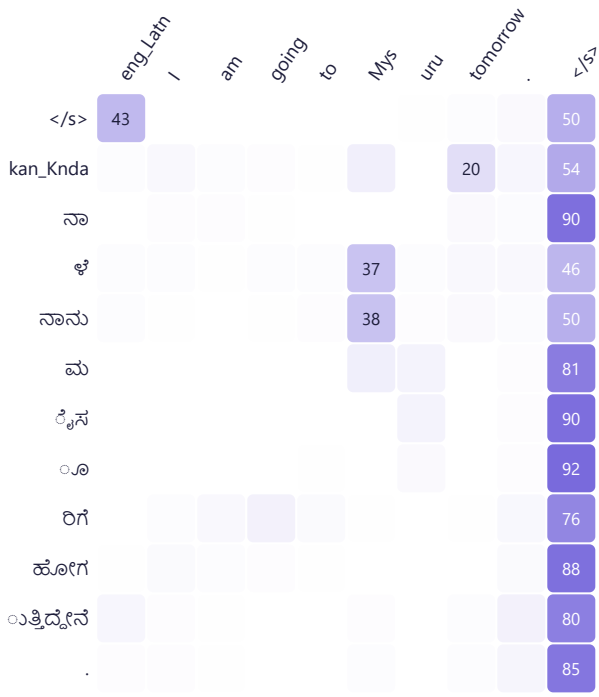
STEP 10: CROSS-ATTENTION, THE BRIDGE

Same attention formula, but the three ingredients come from two places: the **query** from the Kannada written so far, the **keys and values** from the encoder memory. This is how the writer "looks at the reader's notes".

$$Q = Y_{\text{dec}}\, W_Q, \quad K = M_{\text{enc}}\, W_K, \quad V = M_{\text{enc}}\, W_V$$

8. The real cross-attention: Kannada looks back at English

Real weights of decoder layer 12 (average over heads, percent), computed while the model processes its own output. Rows are the Kannada pieces the decoder is working on, columns the English pieces in the encoder memory.



Cross-attention, decoder layer 12. The first two rows are the start tokens (</s>, kan_Knda).

Which English word does each Kannada piece look at most?

Kannada piece (being predicted)	most-attended English word	weight on markers
ನಾ	tomorrow (20%)	56%
ಳೆ	tomorrow (3%)	90%
ನಾನು	Mys (37%)	48%
ಮ	Mys (38%)	51%
ೈಸ	Mys (10%)	81%
ೂರ	uru (7%)	90%
ರಿಗೆ	uru (3%)	93%
ಹೋಗ	going (9%)	76%
ುತ್ತಿದ್ದೇನೆ	I (3%)	89%
.	. (8%)	85%
</s>	. (6%)	86%

HONEST READING

Attention often rests heavily on the end marker; only the share left over is informative, and averaging 16 heads blurs it. In the table, the two pieces of ನಾಳೆ (“tomorrow”) give “tomorrow” the top share among the real words, and the place-name pieces (ಮ, ೈಸ, ೂರ, ರಿಗೆ) look at “Mys” and “uru”, as a human would hope. Other rows are less intuitive (for example ನಾನು, “I”, looks mostly at “Mys”). That is what the model really does: attention weights are evidence of where information can flow, not a complete explanation.

9. Steps 12 and 13: from vectors to the next word

IN PLAIN WORDS

The decoder's final vector for the last position is turned into one score for every token in the vocabulary (256,206 of them) by a linear layer; **softmax** converts the scores into probabilities that sum to 1. The most probable token is chosen (greedy decoding), added to the decoder input, and the loop runs again.

THE MATHS

$$z = W_{out}\, h_t \quad p_k = e^{z_k} / \Sigma_j e^{z_j} \quad \text{next token} = \text{argmax}_k\, p_k$$

W_{out} is 1024 × 256,206; in NLLB it shares its values with the token embedding table.

The real decoding loop for our sentence

turn	last tokens fed in	chosen (probability)	2nd choice	3rd choice
1	</s> kan_Knda	**ನಾ** 57.7%	ನಾನು 35.4%	ನನಗೆ 0.3%
2	</s> kan_Knda ನಾ	**ಳೆ** 95.1%	ಳೆಯ 3.0%	ಳ 0.7%
3	</s> kan_Knda ನಾ ಳೆ	**ನಾನು** 56.2%	ಮ 16.6%	ಮೈ 2.4%
4	kan_Knda ನಾ ಳೆ ನಾನು…	**ಮ** 69.2%	ಮೈ 17.2%	ಮು 1.4%
5	ನಾ ಳೆ ನಾನು ಮ…	**ೈಸ** 96.6%	ಿಸ 1.0%	ಸ 0.6%
6	ಳೆ ನಾನು ಮ ೈಸ…	**ೂ** 42.1%	ೂರು 38.8%	ೂರ 7.0%
7	ನಾನು ಮ ೈಸ ೂ…	**ರಿಗೆ** 79.3%	ರು 9.3%	ರಿನ 1.7%
8	ಮ ೈಸ ೂ ರಿಗೆ…	**ಹೋಗ** 69.7%	ಹೊರ 4.3%	ಹೋಗುವ 4.3%
9	ೈಸ ೂ ರಿಗೆ ಹೋಗ…	**ುತ್ತಿದ್ದೇನೆ** 46.6%	ುತ್ತೇನೆ 27.1%	ಲ 5.8%
10	ೂ ರಿಗೆ ಹೋಗ ುತ್ತಿದ್ದೇನೆ…	**.** 86.9%	</s> 2.0%	. 0.6%
11	ರಿಗೆ ಹೋಗ ುತ್ತಿದ್ದೇನೆ ….	**</s>** 90.1%	. 0.0%	- 0.0%

Pieces that begin with a vowel sign are shown with a dotted circle by some fonts; they are fragments of a Kannada word, not separate letters.

Result: **ನಾಳೆ ನಾನು ಮೈಸೂರಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ.** ("Tomorrow I am going to Mysuru.")

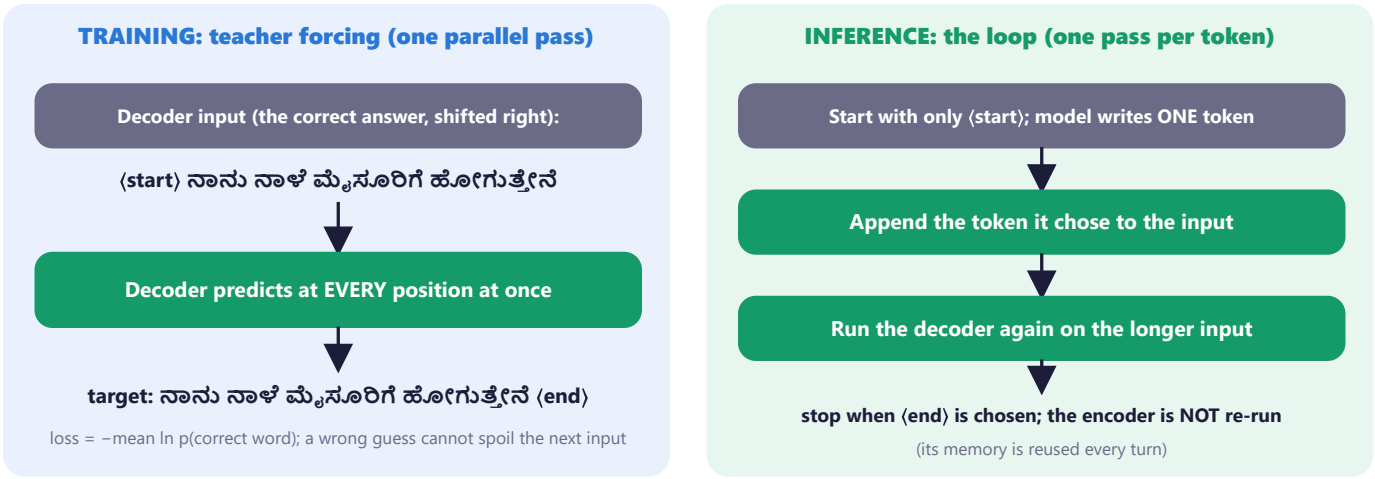
REAL-WORLD PICTURE

The writer at the bus stand pens one syllable at a time. At turn 1 she is 58% sure the sentence starts "ನಾ…" (the start of ನಾನು, "I") and 35% sure it starts with the full word ನಾನು; at turn 2 she is 95% sure the piece after ನಾ is ಳೆ (ನಾಳೆ, "tomorrow"). Each choice is then fed back in, so the next prediction depends on everything written so far.

WHY THE MODEL PUT "TOMORROW" FIRST

Kannada word order is flexible. Both "ನಾನು ನಾಳೆ ಮೈಸೂರಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ" and "ನಾಳೆ ನಾನು …" are correct; a model trained on real text picked the one it found most probable. Beam search or sampling could pick the other. Pieces such as ಮ + ೈಸ + ೂ + ರಿಗೆ spell ಮೈಸೂರಿಗೆ ("to Mysuru") one fragment at a time.

10. Training versus using the model



TEACHER FORCING

During training we know the correct translation, so we feed it (shifted by one) to the decoder and ask for the next word at **every position at once**. This is fast, and one wrong guess cannot spoil the later inputs. During real use there is no answer to feed, so the decoder must use its **own** previous words, one turn at a time.

THE LOSS AND THE UPDATE

$$L = -(1/T)\,\Sigma_t \ln p_t(y_t) \quad \text{then backpropagation and an optimiser step } w \leftarrow w - \eta \cdot \mathrm{Adam}(\partial L/\partial w)$$

At training time the causal mask (step 9) is what makes the parallel pass legitimate: each position still sees only the earlier words. At inference, caching the keys and values of earlier tokens avoids recomputing them every turn; the encoder memory is computed once.

Cost and scale

Quantity	Our example	Why it matters
Encoder passes	1	Source never changes
Decoder passes	11 (one per output token)	Longer answers cost more turns
Self-attention comparisons per layer	10 × 10 = 100 (encoder)	Grows with the square of sentence length
Parameters	615,073,792	Mostly in the feed-forward layers and the 256,206-token embedding table

11. Cheat sheet

Part	Reads	Mixes information by	Produces
Encoder layer	all source words at once	self-attention (all ↔ all), then feed-forward	refined vector per source word
Decoder: masked self-attention	target words so far	attention to earlier target words only	context of what is already written
Decoder: cross-attention	query: target; keys/values: encoder memory	attention from target to source	the source information relevant to the next word
Feed-forward	each position alone	two linear layers with ReLU	per-word processing
Add & LayerNorm	sub-layer input and output	skip connection, then rescale	stable, trainable deep stacks
Linear + softmax	last decoder vector	one score per vocabulary token	probabilities for the next token

Questions people ask

Why "Q, K, V"?

Query = what I am looking for, Key = what I offer to be found by, Value = what I give if chosen. Matching Q against K decides how much of each V is taken.

Why divide by $\sqrt{d_k}$?

Dot products of long vectors grow large and push the softmax to near 0/1, which stalls learning. Dividing keeps the scores moderate.

Why skip connections?

They give the signal and the gradient a direct path through the stack, so 12 layers can train.

Is the decoder slower?

Yes: one pass per output token. The encoder reads the whole input in one parallel pass.

ONE LINE TO REMEMBER

The encoder turns the source sentence into notes (once). The decoder writes the answer one token at a time, each time consulting its own text so far (masked self-attention) and the notes (cross-attention), then choosing the most probable next token.

Sources: Vaswani et al., "Attention Is All You Need" (2017). Real tensors: NLLB-200 distilled 600M, Meta AI (research and education use). Companion pages in this repository: Transformers_Explained_Overview.html and NMT_Pipeline_Explained.html.